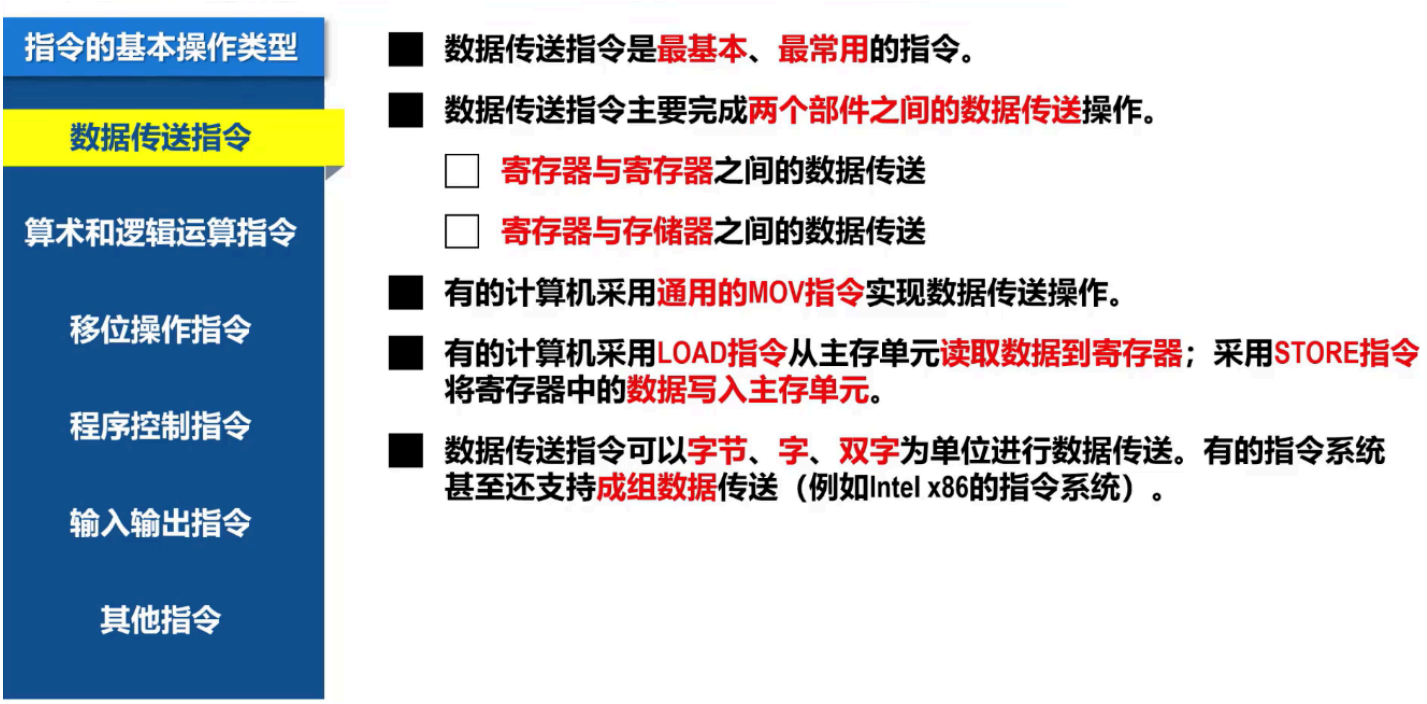
5.8 指令的操作类型

考纲定位：计算机组成原理 · 第 5 章 指令系统（5.8） | 重要度 ★★★★ | 常见考法：选择题（指令分类、转移/调用/返回/中断对比、CF/OF、CMP/TEST）

一句话核心：按“数据传送—算术逻辑—移位—程序控制—I/O—其他”分类；程序控制类的核心是“改 PC 与保存返回地址”

图旁说明：课件图保留为教学主体；文字只补分类判据、四方对比与设错，不重复图内定义与例子

一、数据传送指令



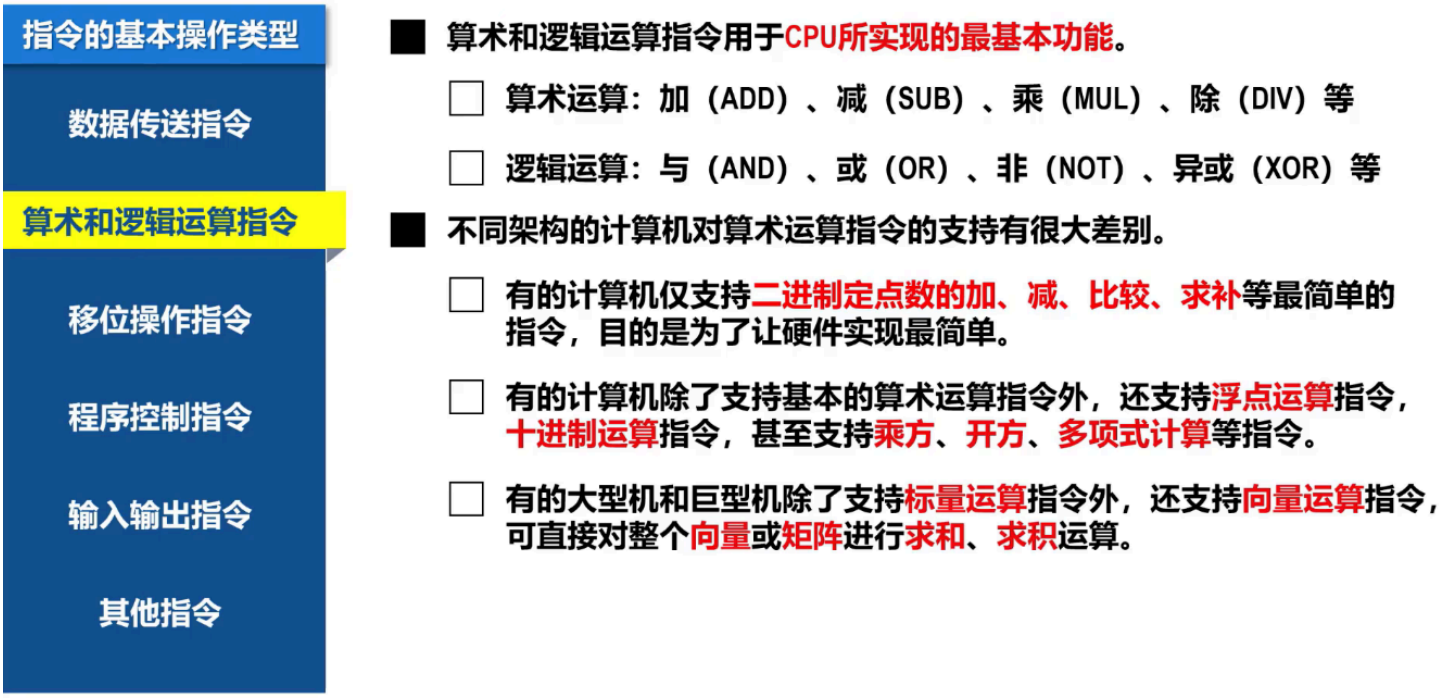
提炼

- 最基本、最常用；完成寄存器↔寄存器、寄存器↔主存、主存↔主存（部分架构）之间的数据搬运
- 典型：MOV、LOAD、STORE；可按字节、字、双字或成组传送
- 数据传送不改变数据内容，通常不影响标志位

易错

- MOV 与 LOAD/STORE 的层次不同：MOV 是通用助记符，LOAD/STORE 是 RISC 的访存专用指令
- 传送方向别写反；`MOV EAX, EBX` 是 EBX→EAX
- 并非所有 MOV 都影响标志位；x86 的 MOV 不影响标志，某些架构的 LOAD 可能影响

二、算术和逻辑运算指令



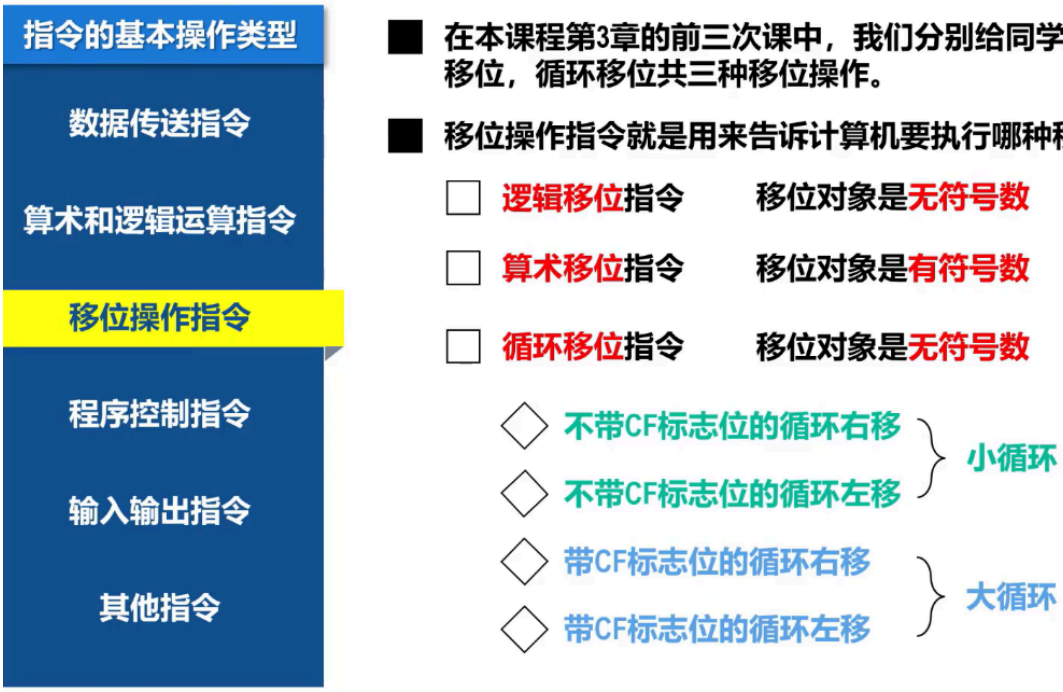
提炼

- 算术：ADD、SUB、MUL、DIV、INC、DEC、CMP
- 逻辑：AND、OR、NOT、XOR、TEST
- 不同架构支持范围差异大：定点、浮点、十进制、向量
- CMP/TEST 只设置标志位，不写回结果

易错

- CMP/TEST 不改变目的寄存器，只影响标志位
- MUL/DIV 的结果寄存器与操作数寄存器可能有隐含约定（如 x86 的 MUL 用 AL/AX）
- 算术指令可能同时影响 CF、OF、SF、ZF；逻辑指令通常使 CF=0、OF=0

三、移位操作指令



提炼

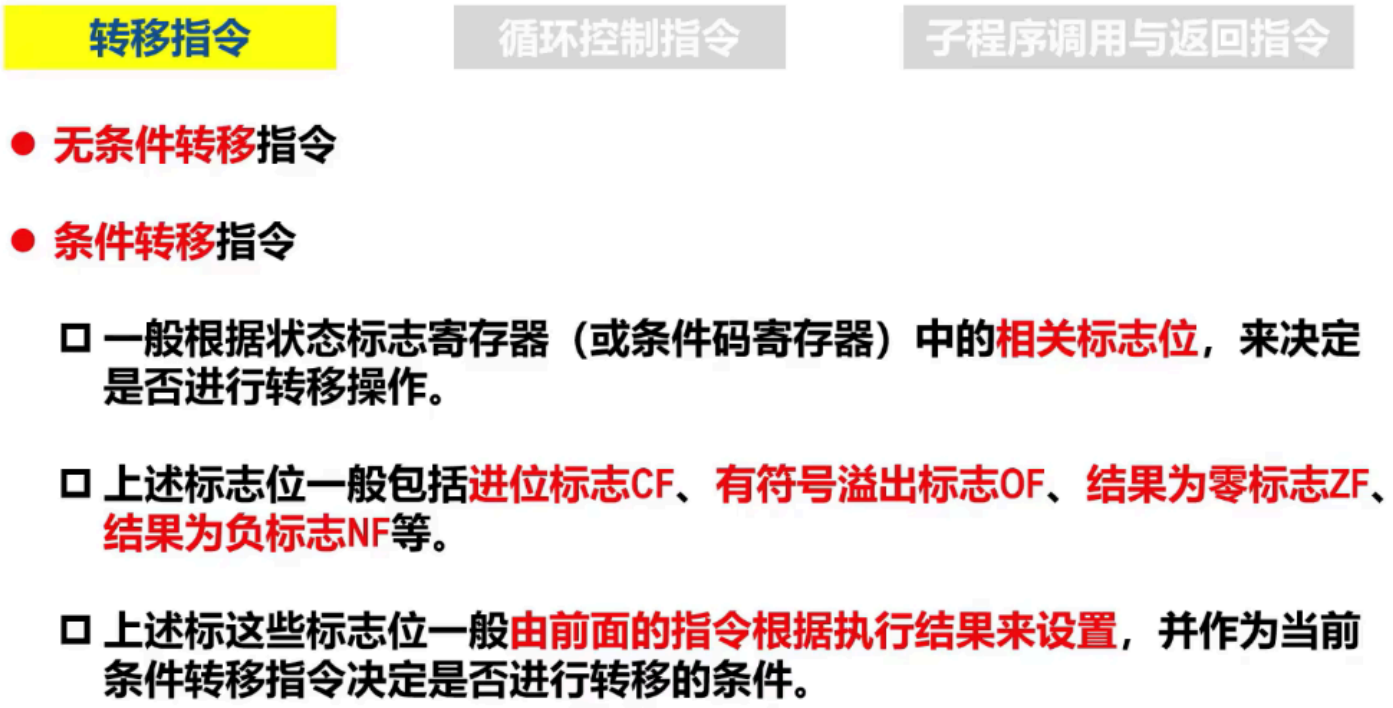
- 逻辑移位：无符号数，空位补 0
- 算术移位：有符号数，右移补符号位，左移补 0
- 循环移位：不带 CF 的“小循环”（寄存器内首尾相接）；带 CF 的“大循环”（CF 参与循环）

易错

- 算术右移补符号位，逻辑右移补 0；两者对负数结果不同
- 循环移位不改变位信息，只改变位置；带 CF 的循环把 CF 当作一位
- 移位次数可能由立即数或 CL 寄存器给出（x86）
- 左移可能改变符号位/溢出，但补码左移低位补 0 仍成立

四、程序控制指令

4.1 转移指令



提炼

- 无条件转移 JMP；条件转移 Jcc
- 条件转移依据状态标志：CF、OF、ZF、SF 等
- 标志由前面的指令设置，条件转移只读不写

易错

- 条件转移不修改标志
- 转移目标通常用相对寻址
- 条件转移的“条件”是标志组合，不是比较结果本身

4.2 循环控制指令

转移指令　　**循环控制指令**　　子程序调用与返回指令

- **为了实现循环程序的执行，循环控制指令一般包括以下功能：**
  - **对循环控制变量的修改**
  - **条件测试和判断**
  - **地址转移**
- **综上所述，循环控制指令实际上是功能更强的转移指令。**

提炼

- 循环控制 = 修改循环变量 + 条件测试 + 地址转移
- 本质是“功能更强的转移指令”
- 典型：x86 的 LOOP 隐含使用 CX/ECX，减 1 后不为 0 则跳转

易错

- LOOP 隐含使用计数寄存器，不要与普通 Jcc 混淆
- x86 的 LOOP 不影响标志，与“先 DEC 再 JNZ”不同
- 循环次数为 0 时，LOOP 的行为要看“先减后判”还是“先判后减”

4.3 子程序调用与返回指令

转移指令　　循环控制指令　　**子程序调用与返回指令**

- 在主程序执行过程中，当需要调用公用的子程序时，可执行子程序调用指令，进入子程序执行。
- 当子程序执行完毕后，可以利用返回指令，从子程序返回到主程序继续执行。
- 子程序调用指令中必须明确给出子程序的入口地址。
- 在主程序中，子程序调用指令的下一条指令的地址称为断点，断点是子程序返回主程序时的返回地址。
- 为了使子程序执行返回指令时能够正确返回主程序，主程序中的子程序调用指令应当具备断点保护的功能，一般情况下，子程序调用指令会将断点（子程序的返回地址）进行压栈保护，而返回指令则从堆栈中取出断点并送入程序计数PC，这样就可使子程序返回到主程序中的断点处继续执行主程序。

提炼

- CALL：把返回地址（下一条指令）压栈，再跳转到子程序入口
- RET：从栈顶弹出返回地址送入 PC
- 子程序调用需要保护断点；返回地址是 CALL 的下一条指令地址
- x86 用栈保存返回地址；MIPS 用 RA 寄存器保存，嵌套调用时需软件压栈

易错

- CALL 与 JMP 的核心区别：CALL 保存返回地址，JMP 不保存
- 返回地址是“CALL 的下一条指令地址”，不是 CALL 本身
- 嵌套调用时栈的 LIFO 保证逐层返回
- x86 的 RET 从栈顶弹出返回地址；MIPS 用 `jr` 跳回，返回地址在 RA 寄存器中
- 参数传递方式（寄存器/栈/内存）与返回地址不是一回事

4.4 转移、调用、返回、中断四方对比



维度	转移 JMP/Jcc	子程序调用 CALL	返回 RET	中断
是否改 PC	是	是	是（从栈/EPC 恢复）	是（硬件送入口）
是否保存返回地址	否	是（压栈）	否（弹出）	是（硬件保存断点）
返回地址来源	无	栈顶	栈顶	EPC/栈
触发方式	指令	指令	指令	外部/异常（异步/同步）
与程序的关系	同一程序内	主程序→子程序	子程序→主程序	任意程序→ISR
是否保护现场	否	由子程序约定	否	ISR 软件保护
典型指令	JMP/Jcc	CALL	RET	中断隐指令 + IRET

提炼

- 四者的共同点都是“改变 PC”
- 区别在于“是否保存返回地址、由谁保存、返回到哪”
- 中断与子程序调用都保存断点，但中断是硬件自动保存，子程序调用是 CALL 指令保存

易错

- 中断返回用 IRET/eret，除了恢复 PC 还要恢复 PSW；子程序返回只需恢复 PC
- 转移指令不保存返回地址，因此不能“返回”
- 中断服务程序的入口由中断向量/硬件提供；子程序入口由 CALL 指令的地址码提供

五、输入输出指令

- **输入输出指令一般简称为I/O指令，用于实现主机与外部设备之间的信息传送。**
  - 主机可向外部设备发出各种控制命令，从而控制外部设备的工作。
  - 主机也可从外部设备端口寄存器中读取外部设备的各种工作状态。
- **当外部设备与主存采用统一编址模式时，则不需要专用的I/O指令，而使用访存指令直接访问外部设备。**
- **除了上述几种类型的指令外，还有一些实现其他控制功能的指令：**
  - 停机指令
  - 等待指令
  - 空操作指令
  - 开中断指令
  - 关中断指令
  - 特权指令
    - ✓ 特权指令主要面向操作系统，用于对系统资源的分配和管理，具有特殊的权限，因而不直接提供给用户使用。
    - ✓ 在多任务、多用户的计算机系统中，必须要有特权指令的支持。
- **在多处理器系统中，还包含有专用的多处理机指令。**

提炼

- I/O 指令实现主机与外设之间的信息传送：发控制命令、读状态、传数据
- 统一编址：I/O 端口占用主存地址空间，可用访存指令访问，不需要专用 I/O 指令
- 独立编址：I/O 端口有独立地址空间，需要专用 IN/OUT 指令

易错

- 统一编址 vs 独立编址是“端口是否占用主存地址空间”，不是“是否有 I/O 指令”
- 统一编址可以用普通访存指令，独立编址必须用 I/O 指令
- 独立编址的端口地址可以与主存地址重叠，靠指令类型区分

跨章：详见 8.3 IO接口

六、其他指令

【2011年 题17】某机器有一个标志寄存器，其中有进位/借位标志CF、零标志ZF、符号标志SF和溢出标志OF，条件转移指令bgt（无符号整数比较大小时转移）的转移条件是（C）。

A. $CF + OF = 1$ 排除　B. $\overline{SF} + ZF = 1$ 排除　C. $\overline{CF} + \overline{ZF} = 1$　D. $\overline{CF} + \overline{SF} = 1$ 排除

解析

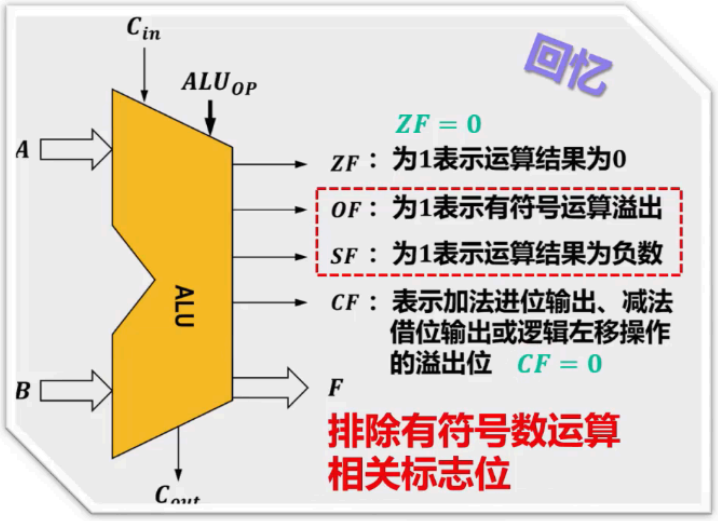
提示

```
mov r0, #10  ; 将数值 10 传送到寄存器r0
mov r1, #5   ; 将数值 5传送到寄存器r1

cmp r0, r1   ; 比较 (r0) 和 (r1)   执行(r0) - (r1)，运算结果影响标志位
bgt greater  ; 如果 (r0) > (r1)，则跳转到标签 greater

; 继续执行其他指令...

greater:
; 这里是跳转目标
```



提炼

- 停机、等待、空操作、开中断、关中断、特权指令
- 特权指令面向操作系统，用于系统资源管理，不直接提供给用户
- 多处理器系统还有多处理器指令

易错

- 特权指令不是“所有系统指令”；NOP 虽然不做事，但占用取指/译码时间
- 开/关中断指令属于特权指令，用户态不能执行
- 停机/等待指令的唤醒条件不同：HLT 等待中断，WAIT 等待条件

七、例题

7.1 例题一

【2018年 题19】减法指令“$sub\ R1,\ R2,\ R3$”的功能为“$(R1) - (R2) \rightarrow R3$”，该指令执行后将生成进位/借位标志$CF$和溢出标志$OF$。若$(R1) = FFFF\ FFFFH$，$(R2) = FFFF\ FFF0H$，则该减法指令执行后，$CF$与$OF$分别为（A）。

A. $CF = 0,\ OF = 0$　B. $CF = 1,\ OF = 0$

C. $CF = 0,\ OF = 1$　D. $CF = 1,\ OF = 1$

考什么：条件转移的标志判据；无符号比较 $a > b$ 对应 $CF = 0$ 且 $ZF = 0$

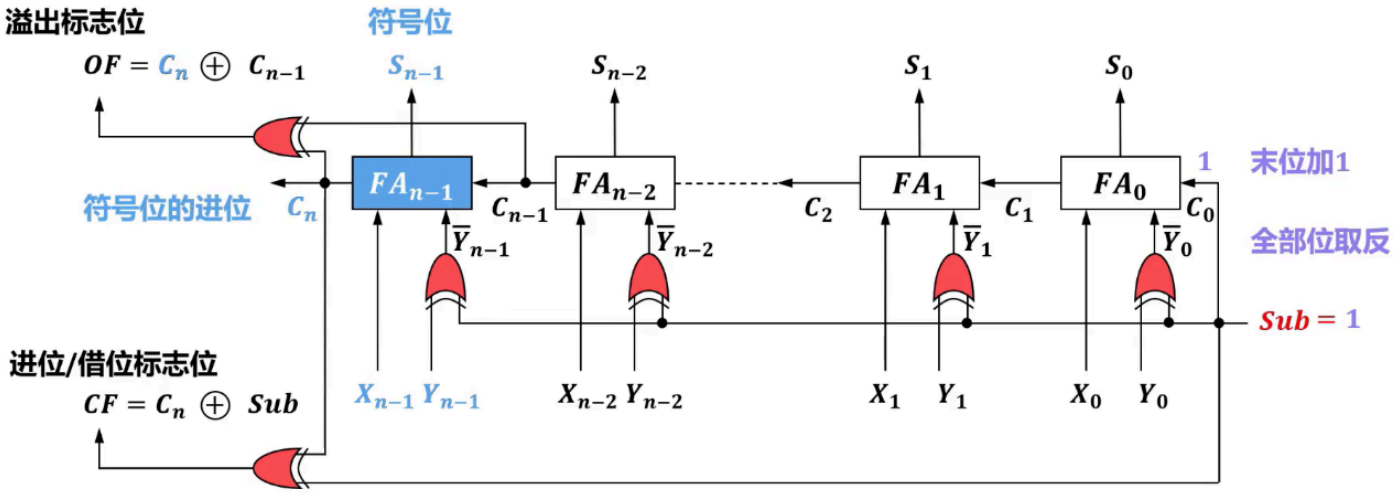
常见错因

- 把无符号比较与有符号比较的标志混用（有符号看 SF/OF，无符号看 CF/ZF）
- 只看 ZF，忘记 CF
- 把“大于”与“大于等于”的条件混同

解题模板

1. 明确比较类型：无符号还是有符号
2. 无符号 $a > b$：做 $a - b$，要求无借位（CF=0）且结果非零（ZF=0）
3. 有符号 $a > b$：看 SF 与 OF 是否相同，且 ZF=0
4. 本题按无符号判据选 C
5. 选项中的“+”号按题目约定理解；本质判据仍是 CF=0 且 ZF=0

7.2 例题二



考什么：sub 指令的 CF/OF 计算；CF 看无符号借位，OF 看有符号溢出

常见错因

- 把 CF 当有符号溢出，或把 OF 当无符号进位
- 减法忘记 CF 的借位极性
- 大数相减时只看符号位，忽略真值

解题模板

1. 先按无符号算减法，判断是否需要借位，得到 CF
2. 再按有符号算，判断结果是否超出范围，得到 OF
3. 本题：$R1 = -1$，$R2 = -16$，$(-1) - (-16) = 15$
4. 无符号 $FFFFFFFFH \geq FFFFFFF0H$，无借位，CF=0
5. 有符号结果 15 在 32 位范围内，OF=0
6. 选 A

CF/OF 结论卡（跨章衔接）

- CF：无符号数越界标志。加法 $CF = C_{out}$；减法 $CF = C_{out} \oplus Sub$，等价于“有借位则 CF=1”
- OF：有符号数溢出标志。$OF = C_n \oplus C_{n-1}$，即进入符号位的进位与移出符号位的进位不同
- 判断口诀
  - 无符号看 CF：加法最高位有进位、减法有借位，则越界
  - 有符号看 OF：正+正得负、负+负得正，则溢出
- 典型反例：$-1 + 1$ 时 CF=1 但结果正确；4 位 $5 + 6$ 时 CF=0 但 OF=1

- 详细推导与多位算例见 3.5 定点数的加减运算——溢出检测；加法器电路见 3.8 串行进位加法器的硬件逻辑实现

知识关联：标志位与算术运算（3.5 定点数的加减运算——溢出检测）； 控制器与条件转移（6.3 指令执行的过程——指令周期）； 中断与子程序调用的区别（6.22 异常与中断——处理过程）； I/O 编址（8.3 IO接口）